ANTECH SOLUTIONS

Task Tracker

Product Documentation · version 1.0.0

Version	**1.0.0**
Date	**2026-10-30**
Prepared by	**Scribe, Antech /ship engineering team**
Prepared for	**CEO Nageshwar**
Contents	**Overview, Quick Start Guide, Specification, API Reference, Architecture Decisions, Test Report, Security Report, Security Audit Checklist**

Contents

1. Overview
    - Run it
    - Build from source
    - Downloadable documentation
    - Documentation
2. Quick Start Guide
    - Before you start
    - Step 1 - Download the files
    - Step 2 - Start the server
    - Step 3 - Use the web app
    - Step 4 - Install the Android app
    - Step 5 - Use the app offline
    - Step 6 - Sync the phone with the server (optional)
    - Step 7 - Explore the API (for developers)
    - Step 8 - Run it for a team (production)
    - Step 9 - Verify your downloads (optional)
    - Troubleshooting
    - Stop and uninstall
3. Specification
    - Scope
    - Domain model - Task
    - User stories and acceptance criteria
    - Android local storage (agreed with Swift)
    - Contract
4. API Reference
    - Task object
    - List tasks - GET /tasks
    - Create a task - POST /tasks
    - Get a task - GET /tasks/{id}
    - Update or create - PUT /tasks/{id}
    - Delete - DELETE /tasks/{id}
    - Sync feed - GET /tasks/changes?since=<instant>
    - Operations

5. Architecture Decisions
    Decisions
    Handoffs
6. Test Report
    Backend (./mvnw test, JDK 21, Spring Boot 4.1.2)
    Android (local JVM unit tests)
    Rejections during the run
    Not covered by automated tests
7. Security Report
    1. Executive summary
    2. Scope
    3. Method
    4. Findings
    5. Controls confirmed
    6. Recommendations
    7. Sign-off
8. Security Audit Checklist
    Findings
    Checklist

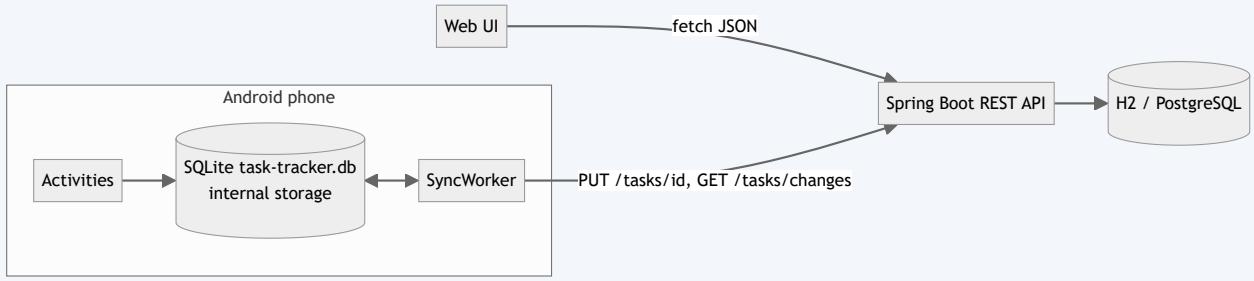
OVERVIEW

Task Tracker v1.0.0

Shipped by the Antech `/ship` agent team for CEO Nageshwar.

> Create tasks with a title, notes, due date and priority, mark them done and filter by status - in the browser or on Android. The Android app stores everything in the phone's internal storage, works fully offline and syncs with the server when you tap **Sync**.

Part	Technology	Location
REST API	Java 21, Spring Boot 4.1.2, Spring Data JPA, H2 (dev) / PostgreSQL (prod), springdoc	`backend/`
Web UI	Vanilla HTML, CSS, JavaScript (`fetch`) served by the API	`backend/src/main/resources/static/`
Android app	Native Java, SQLite in internal storage, `HttpURLConnection` sync	`android/`
Shipment	Executable JAR + APK	`dist/` and the GitHub Release `task-tracker-v1.0.0`



Run it

Executable JAR (Java 17+ required, 21 recommended)

```
java -jar dist/task-tracker-1.0.0.jar
```

Open http://localhost:8080 for the web UI and http://localhost:8080/swagger-ui.html for the API docs. Data is kept in `./data/` (H2 file database) in the folder you start it from.

Production (PostgreSQL)

```
SPRING_PROFILES_ACTIVE=prod DB_URL=jdbc:postgresql://db:5432/tasks DB_USERNAME=tasks
DB_PASSWORD=... \
APP_CORS_ALLOWED_ORIGINS=https://tasks.example.com JPA_DDL_AUTO=update java -jar dist/task-tracker-1.0.0.jar
```

Variable	Default	Purpose
`PORT`	`8080`	HTTP port
`SPRING_PROFILES_ACTIVE`	`dev`	`dev` = H2 file DB, `prod` = env-configured database
`DB_URL`, `DB_USERNAME`, `DB_PASSWORD`	-	Database connection (`prod`)
`APP_CORS_ALLOWED_ORIGINS`	`http://localhost:8080`	Comma-separated origins allowed to call the API
`JPA_DDL_AUTO`	`update` (dev), `validate` (prod)	Schema management. Use `update` on the first prod start

Android APK

Install the APK from the GitHub Release (`task-tracker-1.0.0-debug.apk`, or `task-tracker-1.0.0.apk` when signing secrets are configured):

```
adb install task-tracker-1.0.0-debug.apk
```

Or copy it to the phone and open it (allow "Install unknown apps" for your file manager). To sync, open the menu, choose **Server URL**, enter your server address, then tap **Sync**. Release builds require `https://`. Debug builds on the emulator default to `http://10.0.2.2:8080`.

Where the data lives: `/data/data/com.antech.tasktracker/databases/task-tracker.db` and private shared preferences. Only the app can read them. The app requests no storage permission, and Android backup and device transfer are excluded, so the data stays on the phone until the app is uninstalled.

Build from source

```
cd backend && ./mvnw test && ./mvnw package     # JAR in backend/target/
cd android && gradle wrapper --gradle-version 8.10.2 && ./gradlew testDebugUnitTest
assembleDebug
```

The Android build needs the Android SDK (`ANDROID_HOME`). Without it, push a tag `task-tracker-v*`: GitHub Actions (`.github/workflows/task-tracker-ci.yml`) builds the JAR and APK and attaches them to the release.

Downloadable documentation

Generated by Scribe in `dist/docs/` and attached to the GitHub Release:

File	Contents
`task-tracker-1.0.0-quickstart.pdf` / `.html`	Step-by-step Quick Start Guide for users
`task-tracker-1.0.0-security-report.pdf` / `.html`	Security Report + audit checklist
`task-tracker-1.0.0-documentation.pdf` / `.html`	Complete manual: overview, quick start, spec, API, decisions, tests, security
`task-tracker-1.0.0-docs.zip`	All of the above plus the Markdown sources

Rebuild after editing the Markdown:

```
python plugins/antech-engineering/skills/ship/scripts/build_docs.py --project
projects/task-tracker --name "Task Tracker" --version 1.0.0 --repo-url
https://github.com/naddanki87/antechsolutions/blob/main/projects/task-tracker
```

Documentation

- docs/QUICKSTART.md - step-by-step Quick Start Guide
- docs/SECURITY-REPORT.md - formal security report
- docs/SPEC.md - requirements and acceptance criteria
- docs/openapi.yaml - API contract; live version at `/v3/api-docs`
- docs/API.md - endpoint reference with `curl` examples
- docs/SECURITY-AUDIT.md - Armor's audit and sign-off
- docs/TEST-REPORT.md - Hawk's test results
- docs/DECISIONS.md - architecture decisions and agent handoffs

- `.ship/events.jsonl` - the full agent event log, viewable in Antech Mission Control

QUICK START GUIDE

Quick Start Guide - Task Tracker v1.0.0

This guide takes you from download to a working Task Tracker on your computer and your Android phone, one step at a time. No programming knowledge is needed.

What you get

Part	What it does
Task Tracker server (`task-tracker-1.0.0.jar`)	Stores tasks and serves the web app at `http://localhost:8080`
Web app	Create, complete, filter and delete tasks in any browser
Android app (`task-tracker-1.0.0-debug.apk`)	The same tasks on your phone. Works without internet, keeps all data in the phone's internal storage, syncs when you tap **Sync**

Before you start

You need:

1. A Windows, macOS or Linux computer with **Java 17 or newer**. To check, open a terminal (Windows: press **Win + R**, type `cmd`, press **Enter**) and run `java -version`. If it says "not recognized" or shows a version below 17, install the free **Eclipse Temurin 21** from https://adoptium.net and open a new terminal.
2. For the mobile app: an **Android phone with Android 8.0 or newer**, and a USB cable or another way to copy a file to the phone.
3. The release files from GitHub: https://github.com/naddanki87/antechsolutions/releases/tag/task-tracker-v1.0.0

Step 1 - Download the files

1. Open the release page above.
2. Under **Assets**, download `task-tracker-1.0.0.jar` (the server) and `task-tracker-1.0.0-debug.apk` (the Android app). `SHA256SUMS.txt` lets you check that the downloads are intact (optional, see Step 9).
3. Create a folder, for example `C:\TaskTracker`, and move `task-tracker-1.0.0.jar` into it.

Step 2 - Start the server

1. Open a terminal in the folder. On Windows: open the folder in File Explorer, click the address bar, type `cmd` and press **Enter**.
2. Run: ``` `bash java -jar task-tracker-1.0.0.jar` `
3. Wait until the last lines include `Started TaskTrackerApplication`. This takes about 10 seconds.
4. Leave this window open. Closing it stops Task Tracker. Your tasks are saved in a `data` folder next to the JAR and are still there the next time you start it.

> If you see `Port 8080 was already in use`, start it on another port instead: `java -jar task-tracker-1.0.0.jar --server.port=8090` and use `8090` instead of `8080` in the next steps.

Step 3 - Use the web app

1. Open a browser and go to **http://localhost:8080**.
2. **Add a task:** type a **Title** (required), optionally choose a **Due date** and a **Priority** (Low, Medium, High) and add **Notes**, then click **Add task**.
3. **Complete a task:** tick the checkbox next to it. The title is struck through. Untick to reopen it.
4. **Filter:** use **All**, **Open** or **Done** above the list.
5. **Delete:** click **Delete** next to a task.

If the page shows an error such as "Title is required", correct the field and click **Add task** again.

Step 4 - Install the Android app

1. Copy `task-tracker-1.0.0-debug.apk` to your phone (USB cable, Google Drive, email to yourself, ...).
2. On the phone, open the file (for example in **Files** > **Downloads**).
3. Android asks to allow installing apps from this source: tap **Settings**, turn on **Allow from this source**, go back and tap **Install**.
4. If Google Play Protect warns about an unknown app, tap **More details** > **Install anyway**. This appears because this test build is not signed for the Play Store.
5. Open **Task Tracker** from the app drawer.

Developers can install from a computer instead: `adb install task-tracker-1.0.0-debug.apk`.

Step 5 - Use the app offline

The app works without any server or internet connection:

1. Tap **Add task**, fill in the title, notes, due date (tap the date button) and priority, then tap **Save**.
2. Tap a task to edit it, tick **Done**, or tap **Delete**.
3. Tick the checkbox in the list to complete a task quickly.
4. Use **All / Open / Done** at the top to filter.

Tasks you have not synced yet show **"not synced"** under the title. They are already saved on the phone.

Where your data is stored: in the app's private internal storage (`/data/data/com.antech.tasktracker/databases/task-tracker.db`). Other apps cannot read it, it is not copied to cloud backups, and it is removed only when you uninstall the app.

Step 6 - Sync the phone with the server (optional)

Sync shares tasks between the phone and the web app.

1. Find your computer's address on your Wi-Fi network. On Windows run `ipconfig` and look for **IPv4 Address**, for example `192.168.1.20`. The phone must be on the same Wi-Fi.
2. The app only sends tasks over secure `https://` connections. The single exception is the Android emulator, which reaches your computer at `http://10.0.2.2:8080` (already preset in this test build). For a real phone, publish the server behind HTTPS (see Step 8), or use a tunnelling tool that gives you an `https://` address for `http://localhost:8080`. A plain `http://192.168...` address is blocked by Android and shows "Offline".
3. In the app, open the menu (⋮, top right) > **Server URL**, enter the `https://` address and tap **OK**.
4. Tap **Sync**. You see "Synced: N sent, M received". Tasks created in the web app now appear on the phone, and phone tasks appear in the web app after you refresh the page.
5. Without a connection you see "Offline - changes are saved on this device". Nothing is lost; tap **Sync** again later.

If the same task was changed in both places, the server's version is kept and the phone updates to match.

Step 7 - Explore the API (for developers)

- Interactive API documentation: **http://localhost:8080/swagger-ui.html**
- Health check: **http://localhost:8080/actuator/health** (shows `"status":"UP"`)
- Example: ``` `bash curl -X POST http://localhost:8080/api/v1/tasks -H "Content-Type: application/json" -d '{"title":"First task","priority":"HIGH"}'` ```

See the API Reference section for every endpoint.

Step 8 - Run it for a team (production)

1. Create a PostgreSQL database and user.
2. Start the server with production settings (one line): ``bash SPRING_PROFILES_ACTIVE=prod DB_URL=jdbc:postgresql://db-host:5432/tasks DB_USERNAME=tasks DB_PASSWORD=your-password APP_CORS_ALLOWED_ORIGINS=https://tasks.example.com JPA_DDL_AUTO=update java -jar task-tracker-1.0.0.jar `` On Windows cmd, set each variable first with set NAME=value, then run java -jar task-tracker-1.0.0.jar`.
3. Put it behind an HTTPS reverse proxy (for example Nginx, Caddy or a cloud load balancer) so phones can sync.
4. After the first start, set `JPA_DDL_AUTO=validate`.
5. Note: v1.0.0 has **no login**. Run it only on a trusted network or behind a proxy that requires sign-in until v1.1 adds authentication (see the Security Report, SEC-4).

Step 9 - Verify your downloads (optional)

On Windows PowerShell:

```
Get-FileHash task-tracker-1.0.0.jar -Algorithm SHA256
```

On macOS/Linux: `shasum -a 256 task-tracker-1.0.0.jar`. The value must match the line in `SHA256SUMS.txt`.

Troubleshooting

Problem	Fix
`'java' is not recognized`	Install Java 21 (Before you start, item 1) and open a new terminal
`Unsupported class file major version` / `UnsupportedClassVersionError`	Java is older than 17. Install Java 21
`Port 8080 was already in use`	Add `--server.port=8090` and use port 8090
Web page does not load	Make sure the terminal from Step 2 is still open and shows `Started TaskTrackerApplication`
Android: "App not installed"	Uninstall an older Task Tracker first, then install again
Android: "Use an https:// address"	Release builds only accept `https://` server addresses (Step 6)

Problem	Fix
Android: "Offline - changes are saved on this device"	Check Wi-Fi and the Server URL; the server must be running, reachable and use `https://` (Step 6)
Android: "Sync failed: ..."	The server rejected a change; open the task, check the title (max 120 characters), save and sync again

Stop and uninstall

- **Stop the server:** press **Ctrl + C** in its terminal, or close the window.
- **Remove server data:** delete the `data` folder next to the JAR.
- **Remove the app:** long-press **Task Tracker** > **Uninstall**. This also deletes all tasks stored on the phone.

SPECIFICATION

Task Tracker - Specification (Spec)

Requirement (CEO Nageshwar): Task tracker: users create tasks with a title, notes, due date and priority, mark them done, and filter by status. The Android app works fully offline and syncs when online.

Scope

- Single-tenant, no login in v1.0.0 (Armor-approved: no personal data beyond task text; auth is a v1.1 item).
- Web UI served by the backend; Android app is local-first with optional sync.

Domain model - `Task`

Field	Type	Rules
`id`	UUID	Server-generated on POST; client-generated on PUT (offline create)
`title`	string	required, 1-120 chars after trim
`notes`	string?	max 1000 chars
`dueDate`	date? (`yyyy-MM-dd`)	optional
`priority`	enum `LOW` , `MEDIUM` , `HIGH`	default `MEDIUM`
`done`	boolean	default `false`
`deleted`	boolean	soft delete flag, only visible in `/changes`
`version`	long	optimistic locking; stale -> 409
`createdAt` , `updatedAt`	ISO-8601 UTC instant	server-managed

User stories and acceptance criteria

- **US-1 Create task.** AC-1: valid POST returns 201, `Location` header and the task with `done=false` , `priority=MEDIUM` when omitted, `version=0` . AC-2: blank title or title > 120 chars returns 400

ProblemDetail with a `title` field error.

- **US-2 List and filter.** AC-3: GET `/tasks` returns a page sorted by `createdAt desc` by default, excluding deleted tasks. AC-4: `?done=true|false` filters by status. AC-5: `size` above 100 is clamped to 100.
- **US-3 View task.** AC-6: GET `/tasks/{id}` returns 200; unknown or deleted id returns 404.
- **US-4 Update / complete.** AC-7: PUT with the current `version` updates and returns 200 with `version+1`. AC-8: PUT with a stale version returns 409. AC-9: PUT on an unknown id creates it (201) - offline create. AC-10: PUT on a deleted task returns 409.
- **US-5 Delete.** AC-11: DELETE returns 204 and the task disappears from list/get; unknown id returns 404.
- **US-6 Sync.** AC-12: GET `/tasks/changes?since=<instant>` returns all tasks (incl. deleted) with `updatedAt > since`, oldest first.
- **US-7 Android offline.** AC-13: the app creates, edits, completes and deletes tasks with no network, storing them in internal storage only. AC-14: "Sync" pushes local changes and pulls server changes; network failure keeps local data and shows "Offline".
- **US-8 Web.** AC-15: the web page lists tasks with a status filter, creates tasks, toggles done and deletes, showing loading, empty and error states.

Android local storage (agreed with Swift)

Table `tasks` in `task-tracker.db` (app-private databases dir) mirrors the model plus `dirty`. Settings (API base URL, last pull time) in private SharedPreferences. No storage permissions, backups excluded.

Contract

See `openapi.yaml`.

API REFERENCE

Task Tracker API reference (Scribe)

Base URL: `http://localhost:8080/api/v1`. JSON in and out. Errors are RFC 7807 `application/problem+json`:

```
{ "type": "https://antech.dev/problems/validation", "title": "Validation failed", "status": 400,
  "detail": "One or more fields are invalid", "instance": "/api/v1/tasks",
  "errors": [ { "field": "title", "message": "must not be blank" } ], "timestamp": "2026-10-30T10:15:30Z" }
```

Problem types: `validation` (400), `malformed-request` (400), `request-error` (other 4xx), `not-found` (404), `conflict` (409), `internal` (500, includes `correlationId`).

Task object

```
{
  "id": "3f2b8c1e-6a1d-4f0e-9a43-0f5c2b7d9e11",
  "title": "Pay invoices",
  "notes": "Supplier batch",
  "dueDate": "2026-11-05",
  "priority": "HIGH",
  "done": false,
  "deleted": false,
  "version": 0,
  "createdAt": "2026-10-30T10:15:30Z",
  "updatedAt": "2026-10-30T10:15:30Z"
}
```

List tasks - `GET /tasks`

Query: `done` (true/false, optional), `page` (0..), `size` (1-100, larger values are clamped), `sort` (default `createdAt,desc`).

```
curl "http://localhost:8080/api/v1/tasks?done=false&size=20&sort=dueDate,asc"
```

`200` `{ "content": [Task...], "page": 0, "size": 20, "totalElements": 3, "totalPages": 1 }` · `400` unknown sort field.

Create a task - `POST /tasks`

Body: `title` (required, max 120), `notes` (max 1000), `dueDate` (`yyyy-MM-dd`), `priority` (`LOW|MEDIUM|HIGH`, default `MEDIUM`).

```
curl -i -X POST http://localhost:8080/api/v1/tasks -H "Content-Type: application/json" \
  -d '{"title":"Pay invoices","dueDate":"2026-11-05","priority":"HIGH"}'
```

`201` + `Location: /api/v1/tasks/{id}` + Task · `400` validation.

Get a task - `GET /tasks/{id}`

```
curl http://localhost:8080/api/v1/tasks/3f2b8c1e-6a1d-4f0e-9a43-0f5c2b7d9e11
```

`200` Task · `404` unknown or deleted · `400` id is not a UUID.

Update or create - `PUT /tasks/{id}`

Body: the create fields plus `done` and `version`. Send the `version` you last read; it is ignored when the id does not exist yet (offline create).

```
curl -X PUT http://localhost:8080/api/v1/tasks/3f2b8c1e-6a1d-4f0e-9a43-0f5c2b7d9e11 -H
"Content-Type: application/json" \
  -d '{"title":"Pay invoices","priority":"HIGH","done":true,"version":0}'
```

`200` updated (version + 1) · `201` created · `409` stale version or task deleted · `400` validation.

Delete - `DELETE /tasks/{id}`

```
curl -i -X DELETE http://localhost:8080/api/v1/tasks/3f2b8c1e-6a1d-4f0e-9a43-0f5c2b7d9e11
```

`204` · `404` unknown. Deletes are soft so sync clients can learn about them.

Sync feed - `GET /tasks/changes?since=<instant>`

Returns up to 500 tasks (including deleted ones) changed after `since`, oldest first. Repeat with the last `updatedAt` until fewer than 500 come back.

```
curl "http://localhost:8080/api/v1/tasks/changes?since=2026-10-30T00:00:00Z"
```

`200` `{ "content": [Task...] }` · `400` missing or invalid `since`.

Operations

- `GET /actuator/health` -> `{"status":"UP"}`
- `GET /v3/api-docs` (OpenAPI JSON) and `/swagger-ui.html`

ARCHITECTURE DECISIONS

Architecture decisions and agent handoffs (Scribe)

The complete, timestamped agent conversation is in `../.ship/events.jsonl` and can be replayed in Antech Mission Control.

Decisions

#	Decision	Why	Owner
D1	UUID ids, client-generated allowed via `PUT /tasks/{id}`	Android creates tasks offline without id collisions	Spec + Swift
D2	Optimistic locking with `version`; stale -> 409, server wins	Simple, lossless conflict rule for two clients	Spec + Byte
D3	Soft delete + `/changes?since=` feed	Sync clients must learn about deletions	Spec + Swift
D4	Sync checkpoint = last server `updatedAt`, not the device clock	Avoids clock-skew gaps	Swift
D5	`markSynced` only clears `dirty` when `updated_at` is unchanged	An edit made during an upload is never lost	Swift
D6	Android uses framework SQLite + `HttpURLConnection`, no Room/Retrofit	Small APK, no third-party code, internal storage only	Swift
D7	No authentication in v1	Single tenant, no personal data; planned for v1.1 (SEC-4)	Spec + Armor
D8	Spring Boot 4.1.2 (Initializr default) + springdoc 3.2.0	Current supported line	Byte
D9	Stable `PageResponse` DTO instead of serializing Spring's `Page`	Contract stays stable across Spring versions	Byte
D10	APK built in GitHub Actions	No Android SDK on the build machine; CI is reproducible	Deployer

Handoffs

1. CEO -> Spec: requirement. Spec -> Byte, Swift, Armor: contract review, all approved (Gate 1).
2. Byte, Pixel, Swift implemented in parallel against `openapi.yaml` (Gate 2).
3. Armor -> Byte: SEC-1, SEC-2 (500 on bad input), SEC-6 (generated password in log). Byte fixed all three and updated the `/ship` skill templates so future products inherit the fixes.
4. Hawk -> Byte: build failure (`PropertyReferenceException` moved in Spring Data 4). Fixed, suite green (Gate 3).
5. Scribe documented from the code; Deployer built, smoke-tested, committed, pushed and released (Gate 4).

TEST REPORT

Test report - Task Tracker v1.0.0 (Hawk)

Result: GREEN - 29 tests run, 0 failed, 0 skipped.

Backend (`./mvnw test`, JDK 21, Spring Boot 4.1.2)

Class	Type	Tests	Result	Covers
`TaskServiceTest`	JUnit 5 + Mockito	7	PASS	AC-1, AC-6, AC-7, AC-8, AC-9, AC-10, AC-11
`TaskRepositoryTest`	`@DataJpaTest`	3	PASS	AC-3, AC-4, AC-12, optimistic version
`TaskApiIntegrationTest`	`@SpringBootTest` + MockMvc	12	PASS	AC-1..AC-12, SEC-1, SEC-2, error leakage, security headers, CORS, OpenAPI
`TaskTrackerApplicationTests`	Context load	1	PASS	-

Android (local JVM unit tests)

Class	Tests	Result	Covers
`TaskValidatorTest`	3	PASS	AC-13 input rules (title, notes, due date, priority)
`TaskJsonTest`	3	PASS	AC-14 sync mapping (versionless offline create, versioned update, deleted flag)

Android tests were run locally with JUnit 4.13.2 and org.json against the pure-Java classes; the full Gradle `testDebugUnitTest` + APK build runs in GitHub Actions (no Android SDK on the build machine).

Rejections during the run

1. Build failure: `PropertyReferenceException` moved to `org.springframework.data.core` in Spring Data 4 - fixed by Byte, re-run green.
2. Armor SEC-6 (generated password in log) - fixed by Byte, suite re-run green, logs verified clean.

Not covered by automated tests

- Android UI flows (activities) and on-device sync - manual check after installing the APK (AC-13, AC-14).
- Web UI (AC-15) - checked manually in a browser during delivery.

SECURITY REPORT

Security Report - Task Tracker v1.0.0

Product	Task Tracker v1.0.0 (REST API, web app, Android app)
Assessment	Static code review, configuration review, automated security regression tests
Assessor	Armor (security agent), report compiled by Scribe
Date	2026-10-30
Commit assessed	`feat/task-tracker`, release tag `task-tracker-v1.0.0`
Verdict	**Approved for release.** 0 Critical, 0 High open. 4 findings fixed, 2 Low risks accepted for v1

1. Executive summary

Task Tracker was reviewed against the OWASP Top 10 (2021) for the API and web app and the OWASP Mobile Top 10 for the Android app. Armor found **four issues during development, all fixed before release**: three caused server errors (HTTP 500) on bad input or exposed a generated password in the startup log, and one removed an unneeded database console component. Each fix has an automated regression test that runs on every build.

Two **low risks are accepted** for v1.0.0 by product scope: the API has **no user login** (single-team use, no personal data), and the **debug Android build** may talk to a local test server over plain HTTP. Adding authentication is the main recommendation for v1.1.

Severity	Found	Fixed	Accepted	Open
Critical	0	0	0	0
High	0	0	0	0
Medium	3	3	0	0
Low	3	1	2	0

2. Scope

Component	In scope
Spring Boot REST API (`backend/src/main/java`)	Controllers, services, repository, validation, error handling, security configuration
Configuration (`application.yml`, `pom.xml`)	Profiles, secrets handling, actuator, CORS, dependencies
Web app (`static/index.html`, `app.js`, `styles.css`)	XSS, script sources, data handling
Android app (`android/app`)	Data storage, permissions, network security, backups, exported components, SQL
Build and CI (`.github/workflows/task-tracker-ci.yml`)	Secrets handling, signing, supply chain

Out of scope: infrastructure hosting, penetration testing of a deployed instance, and the Android UI on physical devices.

3. Method

1. Contract review in the architecture phase (data model, endpoints, sync design).
2. Line-by-line review of all changed files against the checklist in the Security Audit Checklist section.
3. Automated scans: secret patterns (`password=`, `secret`, `api_key`, private keys) across Java, YAML, JS, XML and Gradle files, and a search for unsafe DOM APIs (`innerHTML`, `eval`, `document.write`).
4. Security regression tests in `TaskApiIntegrationTest`: `badInputNeverReturns500`, `errorsDoNotLeakInternals`, `securityHeadersAndCors`, run on every CI build.
5. Startup log inspection for credentials.

4. Findings

SEC-1 - Unknown sort field caused a server error (Medium, fixed)

- **Where:** `GET /api/v1/tasks?sort=<field>`
- **Issue:** an unknown field name (for example `sort=bogus,asc`) reached the catch-all error handler and returned HTTP 500. Repeated 500s hide real faults, add noise to monitoring and reveal that input reaches the data layer unchecked.
- **Fix:** `GlobalExceptionHandler` maps `PropertyReferenceException` to `400 malformed-request`.
- **Verification:** `badInputNeverReturns500` asserts 400; smoke test on the shipped JAR returned 400.

SEC-2 - Missing or invalid parameters caused server errors (Medium, fixed)

- **Where:** `GET /api/v1/tasks/changes` without `since`, unsupported content types and other framework errors.
- **Issue:** these fell through to the catch-all handler and returned 500 instead of a 4xx status.
- **Fix:** `MissingServletRequestParameterException` returns 400, and every Spring MVC error with a 4xx status (415, 405, 406...) keeps that status with a generic message.
- **Verification:** `badInputNeverReturns500` covers missing `since`, invalid `since`, non-UUID id, malformed JSON, unknown enum value and `text/plain` (415).

SEC-6 - Generated password written to the startup log (Medium, fixed)

- **Where:** application startup.
- **Issue:** Spring Boot created a default in-memory user and printed `Using generated security password: ...` to the log. Credentials in logs can leak through log shipping and support tickets.
- **Fix:** `SecurityConfig` declares an empty `InMemoryUserDetailsManager`, so no default user or password exists.
- **Verification:** test logs searched after the fix: no generated password present.

SEC-3 - Unused database console dependency (Low, fixed)

- **Where:** `pom.xml` (`spring-boot-h2console`, added by the project generator).
- **Issue:** a web database console is a common attack target if accidentally enabled.
- **Fix:** dependency removed; `spring.h2.console.enabled: false`.

SEC-4 - No authentication in v1.0.0 (Low, accepted)

- **Where:** all `/api/v1/**` endpoints.
- **Issue:** anyone who can reach the server can read and change tasks.
- **Why accepted:** single-team scope agreed in the specification; tasks contain no personal or financial data.
- **Mitigation now:** run on a trusted network or behind a reverse proxy that requires sign-in (Quick Start, Step 8).
- **Planned:** login and per-user ownership checks in v1.1.

SEC-5 - Debug Android build allows HTTP to local test hosts (Low, accepted)

- **Where:** `android/app/src/debug/res/xml/network_security_config.xml`.
- **Issue:** the debug build may send traffic unencrypted to `10.0.2.2` (emulator host) and `localhost`.
- **Why accepted:** debug-only and limited to the two loopback test hosts; release builds allow HTTPS only, and the in-app Server URL setting rejects `http://` in release builds.

5. Controls confirmed

Area	Control
Input validation	Bean Validation on every request body (`@NotBlank`, `@Size`), typed UUID, date, instant and enum binding
Injection	Spring Data derived queries only; Android SQL uses `ContentValues` and `?` arguments
Error handling	RFC 7807 responses without stack traces, exception names or framework messages; 500s get a correlation id in the server log
Transport and headers	CSP `default-src 'self'`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`
CORS	Explicit allow-list from `APP_CORS_ALLOWED_ORIGINS`; other origins get 403 (tested)
Secrets	None in the repository; production database and APK signing keys come from environment variables and GitHub secrets
Exposure	Only static files, API docs, `/actuator/health` and `/actuator/info`, and `/api/v1/**`; everything else denied
Abuse limits	Page size capped at 100, sync feed capped at 500 rows, field length limits
Web XSS	All data rendered with `textContent`; no inline scripts or event handlers; no third-party scripts
Mobile storage	SQLite and settings in app-private internal storage; no storage permissions; backups and device transfer excluded
Mobile components	Only the launcher activity is exported; release builds minified with debug logs stripped

6. Recommendations

Priority	Recommendation	Target
High	Add authentication (for example OAuth2/OIDC or session login) and per-user task ownership	v1.1
Medium	Add rate limiting at the reverse proxy for `/api/v1/**`	Deployment
Medium	Configure APK release signing (`ANTECH_KEYSTORE_*` secrets) before wider distribution	Next release
Low	Add dependency vulnerability scanning (for example OWASP Dependency-Check or GitHub Dependabot) to CI	Next sprint

Priority	Recommendation	Target
Low	Add Flyway migrations and run production with `JPA_DDL_AUTO=validate`	v1.1

7. Sign-off

Armor approves Task Tracker v1.0.0 for release: no Critical or High findings are open, all Medium findings are fixed and covered by automated tests, and the remaining Low risks are documented and accepted within the agreed scope. The detailed per-control checklist follows in the Security Audit Checklist section.

SECURITY AUDIT CHECKLIST

Security audit - Task Tracker v1.0.0 (Armor)

Result: SIGNED OFF - 0 Critical, 0 High open. 4 findings fixed, 2 Low accepted.

Findings

ID	Severity	Location	Issue	Status
SEC-1	Medium	`GET /api/v1/tasks?sort=<unknown>`	Unknown sort property hit the catch-all handler and returned 500	Fixed - `PropertyReferenceException` -> 400; regression test `badInputNeverReturns500`
SEC-2	Medium	`GET /api/v1/tasks/changes` without `since`; other Spring MVC errors	Returned 500 instead of 4xx	Fixed - `MissingServletRequestParameterExcepti` -> 400, any `ErrorResponse` 4xx keeps its status (415, 405, 406...)
SEC-3	Low	`pom.xml`	Initializr added `spring-boot-h2console`	Fixed - dependency removed, console disabled
SEC-6	Medium	Startup log	Spring Boot's generated default user printed `Using generated security password` to the log	Fixed - empty `InMemoryUserDetailsManager` bean; verif no password in test logs
SEC-4	Low	API	No authentication in v1 (single tenant, no personal data)	Accepted by Spec/CEO scope - planned f v1.1
SEC-5	Low	Android	Debug build allows cleartext to `10.0.2.2` / `localhost`	Accepted - debug-only `src/debug` netw config; release is HTTPS-only

Checklist

Check	Result	Evidence
A01 Access control	N/A v1 (SEC-4)	`SecurityConfig`: only static files, docs, health and `/api/v1/**` exposed; everything else `denyAll`
A02 Crypto	PASS	No credentials stored; HTTPS required for Android release
A03 Injection	PASS	Spring Data derived queries only; `@Valid` on every body; UUID/`Instant`/enum binding rejects bad input with 400
A04 Insecure design	PASS	`@Size` limits, page size capped at 100, `/changes` capped at 500 rows
A05 Misconfiguration	PASS	No stack traces/messages in errors (`errorsDoNotLeakInternals`), actuator only `health,info`, CORS allow-list (`securityHeadersAndCors`)
A06 Components	PASS	Spring Boot 4.1.2, springdoc 3.2.0, no unused dependencies
A08 Integrity	PASS	CI uses pinned action major versions, no `curl | sh`
A09 Logging	PASS	500s logged server-side with correlation id; client gets generic detail
S1 Secrets	PASS	Secret scan of `*.java, *.yml, *.js, *.xml, *.gradle`: none. Prod DB and keystore come from env vars
S2 Headers	PASS	CSP `default-src 'self'`, `nosniff`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`
S3 CSRF	PASS	Stateless JSON API without cookies; CSRF disabled by design
Web XSS	PASS	No `innerHTML`/`eval`/`document.write`; all data rendered via `textContent`
Mobile storage	PASS	SQLite + SharedPreferences in app-private internal storage only; no storage permissions; `allowBackup=false`; backup/extraction rules exclude all data
Mobile network	PASS	Release `cleartextTrafficPermitted=false`; base URL setter rejects non-HTTPS in release
Mobile SQL	PASS	`ContentValues` and `?` args only
Exported components	PASS	Only the launcher activity is exported